

InsightDesk: Self-Serve Customer Support with Safe Escalation

Problem Statement and Participant Guide

Venue	Netaji Subhas University of Technology (NSUT), Delhi
Date	6 October 2026
Participation	Teams of up to 4 members
Build window	One full day
Organised by	HCLTech Data & AI Practice
Document version	1.0 (for participants)

Read Sections 1 to 3 before you write any code. Sections 4 to 7 are the contract your system must meet, and Section 9 explains exactly how you will be judged.

1. About this hackathon

This hackathon is part of HCLTech's hiring for the **AI Engineer** role. We are not looking for the most impressive-looking demo. We are looking for people who can engineer AI systems that are **accurate, grounded, safe and measurable**, and who can explain every design decision they made.

Item	Details
Use case	Use case 2: InsightDesk, self-serve customer support with safe escalation
Team size	Up to 4 members. You build as a team; hiring decisions are made individually (see Section 9.4)
Build window	One full day, ending in a code freeze
Judging	30 minutes per team: 10 min demo, 10 min live testing by judges, 10 min Q&A
Your data	A knowledge base you build for the fictional product CloudFlow, plus synthetic customer-account data you generate with an LLM

The principle behind the scoring

The **simplest architecture that meets the requirements** scores highest. Add agents, tools and complexity only where a requirement genuinely needs them, and be ready to explain why.

Escalating everything is as much a failure as answering everything. The skill being tested is **knowing when the system can answer safely and when it must hand off**.

1.1 Day schedule

Exact clock times will be announced at kickoff.

Block	Duration	What happens
Kickoff	30 min	Briefing, questions on this guide, environment check
Build block 1	About 3.5 hours	Suggested target: knowledge base ingested and one cited answer returned through the API
Checkpoint	15 min	Mentors confirm your environment works. Mentors answer clarifying questions; they do not design or debug your solution
Build block 2	About 4 hours	Critic, escalation, tools, evaluation, packaging, documentation
Code freeze	—	Push your final commit and tag it final. Commits after the freeze are ignored
Judging	30 min per team	Slots announced at kickoff. Your system must be running at the start of your slot

2. The problem

Customer support teams are overwhelmed by repetitive "how do I..." tickets that already have answers in the documentation. At the same time, customers do not want a bad chatbot. They want **either a correct answer or a quick handoff** to a human who already has the full context.

A useful support assistant must avoid confident but unsupported answers, must not promise things only a human can approve (refunds, credits, account changes), and must transfer context cleanly when it cannot respond safely.

Your mission

Build an agentic support system for CloudFlow, a fictional SaaS workflow-automation product. It takes a customer query, retrieves matching help articles and resolved tickets, attempts a **grounded answer with citations**, **critiques its own answer** for hallucination and risk, and then either responds or **hands off to a human with a complete context packet**.

2.1 The kinds of requests it must handle

Type	Example	What a good response needs
How-to	"How do I export my workflow run history?"	Steps from the help article for the customer's product version, cited
Troubleshooting	"My Salesforce step fails with error CF-503."	The documented fix, cited. Outdated workarounds in old tickets must not override current documentation
Account and usage	"Why are my API calls failing with 429 errors?"	Usage and plan limits from tools, explained against the cited rate-limit article
Billing and refunds	"I want a refund for this month."	Eligibility checked by a tool; no promise of a refund ; handoff to billing with the evidence
Frustrated customer	"Third time writing. You charged me twice. Get me a manager."	Immediate escalation with a complete handoff bundle and a calm, honest reply
Sensitive action	"I forgot my password, send the reset link here."	Trigger the reset workflow through a tool; never show a token or link in chat
Not covered or out of scope	"Does CloudFlow integrate with SAP Ariba?" (not in the docs) / "Write me a poem."	No invented answer, offer a handoff, or politely decline out-of-scope requests

3. Functional requirements

These requirements describe **behaviour**. A suggested set of agent roles is given in Section 5, but how you meet the requirements is your design decision, and you will be asked to justify it.

R1 Intent and urgency classification

Classify every request with **structured output**: intent (how_to, troubleshooting, account, billing, complaint, security, out_of_scope), urgency, sentiment, product version (if stated or known), PII detected, tools needed and confidence. Validate the output and handle invalid model responses.

R2 Grounded retrieval

Retrieve from both **help-center articles** and **resolved tickets**. Answers must come from retrieved content and tool results, not from the model's general knowledge.

R3 Citations

Every factual answer cites its sources: **source ID, section, product versions covered and last-updated date**. Judges will open cited sources and check they support the answer.

R4 Self-critique

A critic step checks every draft for **groundedness, coverage, PII and policy risk**, returning a structured decision: answer, revise or escalate. Allow **at most one revision** before escalating.

R5 Escalation and handoff

Escalation decisions follow the **Escalation Policy in Annex A**, applied in code over the critic's structured output and tool results. Every escalation produces a **handoff bundle** with the fields in Annex D, and a customer-facing message that promises nothing a human has not approved.

R6 Versions and source precedence

Apply the **Source Precedence Policy in Annex A**: match the customer's product version, honour deprecations and their dates relative to as_of_date, and never let an old resolved ticket override current documentation. Record conflicts in the audit trail.

R7 Deterministic tools

Account facts, usage against plan limits, invoice status, refund eligibility and platform status must come from **tools over SQLite and the policy registry**, never from LLM arithmetic or judgement. The LLM chooses tools and explains results. Refunds, credits and account changes are **never executed** by the system; they are handed to a human.

R8 Authorisation

The customer's account comes **only** from the X-Account-Id header, never from the message text. Refuse requests for another account's data.

R9 PII and secrets

Detect and redact PII and secrets (email addresses, phone numbers, card numbers, API keys) in **responses, handoff bundles and logs**. Never echo a secret back. Password resets go through a tool that never reveals the token or link.

R10 Untrusted content

Customer messages and ticket text are **data, not instructions**. Text inside them must never change the system's behaviour or policies.

R11 Response typing and auditability

Every response has an `answer_type` (Section 6.2) and a `trace_id` with an audit record: intent, sources retrieved, tools invoked, critic scores, escalation reasons, model, latency and tokens. Do **not** expose chain-of-thought.

R12 Live ingestion

New articles and tickets can be ingested through POST /ingest while the system is running, and are used immediately. **Judges will do this during judging with content you have not seen.**

R13 Evaluation

Build an evaluation set and report measured results (Section 7). A system with no evaluation is incomplete.

4. Your data

Option A — Public Data Sources

- MS MARCO QnA Dataset — Real Bing user questions with answers — great for the query side.
- Customer Support on Twitter (Kaggle) — 3M+ real customer support interactions from major brands. Excellent for ticket-style data.
- Stripe Documentation — Comprehensive, well-structured product docs — pull a subset into your help-center corpus.
- Twilio Help Center — Public help-center articles in clear FAQ format.
- GitHub Discussions API — Pull real Q&A threads from popular OSS projects.
- Stack Overflow Data Dump — Full historical archive of Q&A — filter by tag for domain relevance.

Option B — Generate Synthetic Data with an LLM

4.1 Knowledge base for CloudFlow

All teams support the same fictional product, **CloudFlow**, so that judges can test every team on equal terms. You build the knowledge base yourselves, mainly with LLM-generated content. A starter prompt is in Annex F; improving it is part of the exercise.

Minimum content

- At least **30 help-center articles** covering getting started, account and billing, API and integrations, troubleshooting (with error codes) and advanced features
- At least **2 product versions** (for example 3.x and 4.x) where some features or steps differ by version

- At least **25 resolved tickets**, including **3 or more angry complaints** that required a human, and **3 or more** whose workaround is outdated or contradicted by current documentation
- At least one **release note with a deprecation** that takes effect on a future date
- Policy articles that define your **refund window, plan limits and escalation SLAs**; these values go into the policy registry (Annex C)

More complex knowledge bases earn more marks: version-specific content, deprecations, error-code tables, long articles, outdated tickets that genuinely conflict with documentation, and several document types.

Using public data sources (optional)

You may use public sources to make your data more realistic, for example: customer-message tone and phrasing from the Customer Support on Twitter dataset (Kaggle), query variety from MS MARCO, or article structure modelled on public help centers (Stripe, Twilio), GitHub Discussions or Stack Overflow.

Rules for data

Check the licence and terms of use of any public source; do not scrape sites against their terms.

Remove or anonymise all real personal data (names, handles, emails) from anything you take from public sources.

Record every document and ticket in the **Source Register** (Annex B), with synthetic: Y for generated content.

4.2 Synthetic customer-account data

Generate customer accounts, usage, invoices and platform status with an LLM, in the **fixed schema in Annex C**. You may add columns or tables, but may not rename or remove the required ones, because judges will load their own test accounts in this exact schema. We assess **how** you use AI to generate structured data: prompt design, schema enforcement, validation and edge-case coverage.

Minimum requirements

- At least **30 accounts** across all four plans (Free, Pro, Business, Enterprise) and all four statuses (active, past_due, suspended, cancelled)
- Edge cases included deliberately: usage **exactly** at a plan limit; one unit **over** it; a failed payment; a duplicate charge; a charge **on the last day** of the refund window and one day after it; a suspended account; an account on an old product version
- A **validation script** that enforces the schema and logical constraints (only card last-4 digits stored; amounts positive; statuses consistent with invoices) and reports violations
- A **loader** that adds data from CSVs in the Annex C schema, for example `python scripts/load_accounts.py --dir test_accounts/`, or an equivalent admin endpoint. Judges will use it

Submit with your data

- The exact **prompts** you used (verbatim) and the model that produced the data
- The **generator script** and the **validation script** with its output
- A one-page **data card** (Annex E)

5. Technology stack (mandatory)

Layer	Required choice	Notes
User interface	Streamlit (recommended) or React	A simple chat view is enough. The UI is scored only for usability
API	FastAPI + Uvicorn, Pydantic v2	Must implement the contract in Section 6
Orchestration	LangGraph (Python)	Agent design is your decision; see 5.1
Vector store	ChromaDB, persisted to disk	Articles and tickets, with metadata
Structured data	SQLite	Accounts, usage, invoices, platform status, policy registry, handoffs
Embeddings	sentence-transformers (all-MiniLM-L6-v2 or BAAI/bge-small-en-v1.5)	Be ready to justify your choice
LLM	Ollama, running locally (for example llama3.1:8b or qwen2.5:7b-instruct)	A cloud LLM may be used only as a fallback behind a configuration switch, disclosed in the README
Packaging	Docker + docker compose	Ollama may run on the host machine; document how to connect
Version control	Git	Commit throughout the day. All members should commit

5.1 Suggested agent roles

This is a **starting point, not a requirement**. You may merge, split or replace roles, or implement some as plain code, as long as you can justify the choice.

Role	Responsibility
Orchestrator	LangGraph supervisor; routes between retrieval, answering, critique and escalation
Intent Classifier	Classifies request type, urgency and sentiment with structured output
Knowledge Agent (RAG)	Retrieves the top-k matching articles and resolved tickets
Answer Composer	Writes a grounded answer with citations
Critic	Checks groundedness, PII and confidence; recommends answer, revise or escalate

Role	Responsibility
Escalator	Builds the handoff bundle for a human agent

A design question you will be asked

Which of these decisions should an LLM make, and which should be plain code? For example: should the final answer-or-escalate decision be made by the critic model, or by code applying the Escalation Policy to the critic's scores?

5.2 Practical tips

- Pull your Ollama model before the event and test structured (JSON) output with it. Local 7-8B models can be unreliable at tool calling and JSON; validate their output and retry or fall back.
- Add a MOCK_LLM=true mode so you can test routing, tools and escalation without the model.
- Get one end-to-end path working early (one article, one question, one cited answer), then add the critic and escalation.
- Commit every hour.

6. API contract (mandatory)

This contract is fixed so judges can run the same live tests against every team.

Endpoint	Purpose	Notes
POST/support	Handle a customer message	Header X-Account-Id identifies the customer. Body: message, optional conversation_id, channel, product_version, as_of_date (YYYY-MM-DD, defaults to today)
POST/ingest	Add an article or ticket while running	Multipart: a Markdown article or a JSON ticket, plus metadata JSON with the Source Register fields (Annex B)
GET/health	Readiness	Status of API, vector store, SQLite and LLM
GET/conversations/{id}	Conversation history	Messages, routes taken and responses
GET/handoffs/{id}	Handoff bundle	The full bundle created at escalation
GET/audit/{trace_id}	Audit record	Full audit record for one response
GET/sources	Source Register	All ingested articles and tickets with metadata
Test-data loader	Load judge test data	CLI script or admin endpoint that loads CSVs in the Annex C schema

6.1 Response format for POST /support

```
{
  "trace_id": "b81e0c44",
  "conversation_id": "C-0192",
  "answer_type": "answered",
  "answer": "In CloudFlow 4.3, open Workflows, select the workflow, then ...",
  "intent": { "type": "how_to", "urgency": "low", "sentiment": "neutral",
    "pii_detected": false, "confidence": 0.92 },
  "citations": [
    { "source_id": "KB-ADV-007", "doc_type": "article", "section": "Steps",
      "product_versions": "4.2+", "last_updated": "2026-09-10" }
  ],
  "tools_invoked": [
    { "tool": "lookup_account",
      "output": { "plan": "Pro", "status": "active", "product_version": "4.3" } }
  ],
  "critic": { "groundedness": 0.94, "coverage": "complete",
    "pii_risk": "none", "policy_risk": "none",
    "decision": "answer", "revisions": 0 },
  "conflicts_detected": [],
  "handoff_id": null,
  "as_of_date": "2026-10-06"
}
```

6.2 Answer types

answer_type	Use when
answered	A grounded answer passed the critic and the escalation policy
clarification_needed	The request is too vague to act on (for example, " it' s not working"); ask a specific question
escalated	The escalation policy requires a human; a handoff bundle was created
not_found	The knowledge base does not cover the question; say so and offer a handoff
refused	The request is not allowed (another account' s data; revealing secrets)
out_of_scope	The request is unrelated to CloudFlow support

7. Evaluation (mandatory)

A demo shows that your system can work. An evaluation shows **how well** it works. Build an evaluation set of at least **25 requests**, each labelled with the expected outcome (answer content and sources, or escalation and reason):

- At least **5** answerable how-to questions and **3** version-specific questions
- At least **3** cases where an outdated ticket conflicts with current documentation
- At least **4** account or billing requests that need tools

- At least **4** requests that **must** escalate (angry complaint, refund, security, explicit request for a human)
- At least **2** messages containing PII or secrets, **2** out-of-scope requests and **2** attempts to access another account

Report these results

Metric	What it measures
Answer correctness	Answers match the expected content for the customer's version
Citation validity	Cited sources exist in retrieval output and support the answer
Retrieval hit rate	The expected article appears in the top-k results
Escalation precision and recall	A confusion table of escalated versus answered against your labels. Over-escalation and under-escalation both count
Critic agreement	How often the critic's groundedness verdict matches a human check on a sample
PII leakage	Number of responses, bundles or log lines exposing PII or secrets (target: zero)
Latency and cost	p50 and p95 latency; LLM calls and tokens per request

State your method. For **higher marks**: compare at least two configurations (for example, chunking strategy, a reranker, critic threshold or prompt) and justify your final choice with the numbers.

8. Deliverables checklist

- Git repository, tagged final, with a **README**: architecture diagram, setup and run steps, sample curl commands, assumptions, limitations and known edge cases
- dockercompose up starts the API, UI and stores (Ollama may run on the host)
- **Source Register** (CSV, Annex B) and the knowledge base itself
- **Policy registry** in SQLite, every value linked to a cited policy article
- **Synthetic data kit**: prompts, generator script, validation script and output, data card
- **Evaluation set and report** (Section 7)
- **Three sample audit records** and **two sample handoff bundles**
- **AI-usage disclosure**: which parts were generated with AI coding assistants and how you verified them
- **Team contribution statement** and a signed **declaration of original work** by every member

9. How you will be judged

9.1 Your 30-minute slot

Time	What happens	How to prepare
0– 10 min	Your demo	Show at least: one cited answer, one tool-based account answer, one escalation with its handoff bundle, and one request where the critic changed the outcome
10– 20 min	Live testing by judges	Judges will ingest new articles and tickets through POST /ingest, load test accounts with your loader, and send requests you have not seen. Make sure these paths work
20– 30 min	Q&A	Questions are directed at individual members , including about parts they did not build. You may be asked to make a small live change

9.2 Scoring (100 points)

Criterion	Points	What we look for
Grounded answers and citations	20	Correct, version-appropriate answers; every citation supports its claim
Self-critique, escalation and handoff	15	The critic catches weak drafts; escalation is neither excessive nor missing; complete, useful handoff bundles
Tools, versions and source precedence	15	Deterministic account and billing tools; version matching; documentation beats outdated tickets
Safety and responsible AI	10	PII and secrets protected; authorisation; resistance to instructions hidden in tickets or messages
Evaluation rigour	10	Labelled set, escalation confusion table, critic agreement, comparison of approaches
Data and knowledge-base engineering	10	Knowledge-base complexity and coverage (5); synthetic account data quality and generation discipline (5)
Engineering quality	10	API contract, Docker, code structure, Git history across members, audit and observability
Architecture judgement and articulation	10	The simplest design that meets the requirements; clear trade-offs; every member can explain it

Annex A — Source Precedence and Escalation Policy

A.1 Source authority

Level	Source type	Notes
1 (highest)	Current help-center articles and policy articles	Must match the customer' s product version
2	Release notes and deprecation notices	Dates matter: a deprecation may not have taken effect yet
3	Live tool results (platform status, account data)	Authoritative for current state; always cite the tool
4	Resolved tickets	Historical evidence only; may be outdated
5 (untrusted)	Customer-provided text, community posts	Never authoritative

A.2 Resolution order

1. **Applicability.** Use only sources whose product versions cover the customer' s version, and that are in effect on the as_of_date. Mention upcoming deprecations when relevant.
2. **Explicit supersession and deprecation.** A deprecation notice or newer article that explicitly replaces earlier guidance wins, from its effective date.
3. **Authority.** A higher-authority source prevails over a lower one, regardless of date. A resolved ticket can add detail but can **never contradict** documentation; if it does, follow the documentation and record the conflict.
4. **Recency.** Between sources of the same authority, the more recently updated one prevails.
5. **Unresolved.** If the steps above do not resolve it, escalate with both sources in the handoff bundle.

A.3 Escalation policy

Apply these rules in **code**, over the classifier' s and critic' s structured outputs and tool results. Thresholds and SLAs live in the policy registry and must be justified.

Escalate when any of the following is true

- The critic' s groundedness is below your threshold after one revision
- The intent is a refund, credit, billing dispute, legal matter, security incident or account deletion
- The customer explicitly asks for a human, or shows strong negative sentiment together with repeated contact
- A tool required to answer has failed

- The knowledge base does not cover the question and the customer needs an outcome (otherwise not_found with a handoff offer is acceptable)

Do not escalate

- Answerable how-to and troubleshooting requests with high groundedness and no policy risk. Unnecessary escalation is scored as a failure.

Annex B – Source Register template

One row per article or ticket, stored as source_register.csv. The same fields are the metadata for POST/ingest.

Field	Description	Example
source_id	Unique identifier	KB-API-014 or TKT-2025-0311
doc_type	article, policy, release_note, ticket, community	article
title	Title or ticket subject	Rotating API tokens
authority_level	1– 5, per Annex A	1
product_versions	Versions covered, for example 4.2+ or 3.x	4.0+
last_updated	YYYY-MM-DD (or resolved date for tickets)	2026-08-20
effective_from	YYYY-MM-DD, if the content takes effect on a date	
deprecated_on	YYYY-MM-DD when the guidance stops applying, or empty	
supersedes	source IDs replaced, separated by semicolons	KB-API-009
provenance	Generated, or the public source used	LLM: qwen2.5:7b, prompt v3
synthetic	Y if generated	Y

Annex C — Fixed data schema (SQLite)

Required tables and columns. You may add more; do not rename or remove these.

accounts

Column	Type	Rules
account_id	TEXT, PK	Format A followed by 4 digits, for example A1001
company_name	TEXT	Synthetic
owner_email	TEXT	Synthetic, @example.com domains only. Treat as PII
plan	TEXT	Free, Pro, Business or Enterprise
status	TEXT	active, past_due, suspended or cancelled
product_version	TEXT	For example 4.3
created_at	TEXT	YYYY-MM-DD

plan_limits

Column	Type	Rules
plan	TEXT, PK	One row per plan
api_rate_limit_per_min	INTEGER	
monthly_workflow_runs	INTEGER	
seats	INTEGER	
support_tier	TEXT	For example standard or priority
monthly_price	REAL	

usage

Column	Type	Rules
account_id	TEXT, FK	Primary key with period
period	TEXT	YYYY-MM
workflow_runs	INTEGER	Runs so far in the period
api_calls_peak_per_min	INTEGER	Highest per-minute rate in the period
seats_used	INTEGER	

invoices

Column	Type	Rules
--------	------	-------

Column	Type	Rules
invoice_id	TEXT, PK	
account_id	TEXT, FK	
amount	REAL	> 0
currency	TEXT	For example USD or INR
charged_on	TEXT	YYYY-MM-DD
status	TEXT	paid, failed or refunded
failure_reason	TEXT	Empty unless failed
card_last4	TEXT	Last four digits only. Never store full card numbers

platform_status

Column	Type	Rules
component	TEXT, PK	For example api, workflow-engine, connectors, billing
status	TEXT	operational, degraded or outage
incident_id	TEXT	Empty if operational
updated_at	TEXT	ISO timestamp

policy_registry

Column	Type	Rules
rule_id	TEXT, PK	For example REFUND-WINDOW-01
description	TEXT	Plain-language description
parameter	TEXT	For example refund_window_days, critic_min_groundedness
operator	TEXT	<=, >= and so on
value	TEXT	
scope_plans	TEXT	ALL or a list of plans
effective_from	TEXT	YYYY-MM-DD
source_id	TEXT	Policy article in the Source Register
source_section	TEXT	

handoffs

Column	Type	Rules
--------	------	-------

Column	Type	Rules
handoff_id	TEXT, PK	
conversation_id	TEXT	Created by your system
account_id	TEXT	
queue	TEXT	
priority	TEXT	For example billing, technical, security
created_at	TEXT	low, normal, high or urgent
bundle_json	TEXT	ISO timestamp
	TEXT	The handoff bundle (Annex D), PII redacted

Required tools

Implement at least these as deterministic tools over the tables above: lookup_account, get_usage, get_plan_limits, get_invoices, check_refund_eligibility, check_platform_status, send_password_reset (mocked; never returns a token or link) and create_handoff.

Reserved for judges: account IDs A9000 to A9999, invoice IDs starting INV-J, and source IDs starting JD-. Do not use them in your own data.

Annex D — Escalated response and handoff bundle

```
{
  "trace_id": "e0a971c2",
  "answer_type": "escalated",
  "answer": "I'm sorry about the duplicate charge. I've passed this to our billing team with the details below. You will hear back within one business day. I can't issue refunds myself.",
  "handoff_id": "H-0042",
  "handoff": {
    "queue": "billing", "priority": "high",
    "intent": "billing_dispute", "urgency": "high", "sentiment": "angry",
    "escalation_reasons": ["billing_dispute", "explicit_human_request", "repeated_contact"],
    "customer_summary": "Customer reports being charged twice on 1 Oct.",
    "evidence": [
      {"tool": "get_invoices", "output": [
        {"invoice_id": "INV-6001", "amount": 49.0, "charged_on": "2026-10-01"},
        {"invoice_id": "INV-6002", "amount": 49.0, "charged_on": "2026-10-01"}]
      },
      {"source_id": "KB-BIL-003", "section": "Duplicate charges"}
    ],
    "attempted_answer": "Explained duplicate charge policy, refund needs approval.",
    "unresolved_questions": ["Approve refund of INV-6002?"]
  },
  "pii_redacted": true
}
```

Annex E — Synthetic data card template

Field	What to write
Purpose	What the data is for and what it must exercise
Generator	Model name and version, temperature, number of calls
Prompts	Link to the verbatim prompts in the repo
Schema enforcement	How you forced valid structure (JSON schema, Pydantic, retries)
Row counts and distributions	Accounts per plan and status; usage and invoice distributions
Edge cases included	Which edge cases, and the account IDs that carry them
Validation results	Checks run, violations found, and how you fixed them
What the LLM got wrong	Errors you caught in generated data and how
Known limitations	Ways the data is unrealistic or incomplete

Annex F — Starter prompt for the knowledge base

Use this as a starting point. It does **not** yet satisfy Section 4.1 (for example, it has no product versions, deprecations, outdated tickets or policy articles). Improving it is part of what we assess.

Generate a customer support knowledge base for a fictional SaaS product called 'CloudFlow' (workflow automation tool). Produce:

1. 30 help-center articles as markdown files covering:

- Getting started (5 articles)
- Account & billing (5 articles)
- API & integrations (8 articles)
- Troubleshooting (8 articles)
- Advanced features (4 articles)

2. 25 resolved support tickets as JSON with:

id, customer_question, intent (how_to/bug/billing/account/complaint), resolution, tags, resolved_at

Make at least 3 tickets be angry complaints requiring human escalation. Output realistic, specific content with version numbers and error codes.